

基础知识学习 - 3. GNN

4.1 内容简介

- 学习Node embedding的概念和建模方法
- 学习（无节点属性）图数据上的几种机器学习任务——Link prediction和Node classification
- 学习经典的GNN模型架构
- 学习GNN模型在不同（有节点属性）图数据任务上的应用
- 学习Embedding可视化的原理和方法

4.2 Node Embedding

【Node Embedding 概念】

- 了解node embedding的概念
- 学习经典node embedding模型 —— Deepwalk, LINE, Node2vec
- 学习（无节点属性）图数据上的link prediction任务和node classification任务
- 论文阅读：
 - DeepWalk: Online Learning of Social Representations [<https://arxiv.org/abs/1403.6652>]
 - LINE: Large-scale Information Network Embedding [<https://arxiv.org/abs/1503.03578>]
 - node2vec: Scalable Feature Learning for Networks [<https://arxiv.org/abs/1607.00653>]

4.3 Graph Neural Networks

【Graph neural networks (GNN)概念与架构】

- 学习几种GNN的经典模型架构（GCN、GAT、Graphsage）：
 - GCN: Semi-Supervised Classification with Graph Convolutional Networks [<https://arxiv.org/abs/1609.02907>]
 - GAT: Graph Attention Networks [<https://arxiv.org/abs/1710.10903>]
 - Graphsage: Inductive Representation Learning on Large Graphs [<https://arxiv.org/abs/1706.02216>]
 - Pooling概念: Understanding Pooling in Graph Neural Networks [<https://arxiv.org/pdf/2110.05292>]
- 归纳几种图任务形式：
 - 半监督节点分类任务
 - 图分类任务
 - 链路预测任务
 - 比较以上几种任务的不同

- 比较以上两种任务的不同
- 学习用经典GNN架构实现半监督节点分类和图分类任务

4.4 降维可视化方法

- 学习经典高维向量降维方法PCA, t-SNE的原理
- 使用降维方法可视化Embedding

4.5 实验内容

注：本实验内容主要基于DGL框架实现，在以下实验内容中如果没有特意说明，请使用

DGL实现。

【Node embedding —— Link prediction】

- 主要目标：基于Node embedding实现图上的link prediction任务
- 数据集：Cora, Citeseer, Pubmed（三个数据集选一个）
- 任务描述：根据给定的图结构数据，获得每个节点的Embedding
- 数据集获取&处理：
 - 使用DGL加载三个数据集，分别为：
 - `dgl.data.CoraGraphDataset()`
 - `dgl.data.CiteseerGraphDataset()`
 - `dgl.data.PubmedGraphDataset()`
 - 按照train/test划分训练/测试集，即将数据集中的边划分成两个集合，两个集合中的边不重合，训练集中要包含所有节点。
- 主实验：
 - 在训练集上训练得到每个节点的Node embedding，可使用的DGL接口有：
 - `dgl.sampling.random_walk()`
 - `dgl.sampling.node2vec_random_walk()`
 - 在测试集上测试Node embedding的质量，评测指标为AUC
 - 比较不同embedding模型在测试集和训练集上的效果差异
- 超参实验：（在不同模型中尝试不同超参带来的测试集上的差异）
 - Deepwalk：
 - 随机游走路径长度
 - 每个节点起始的路径数量
 - node2vec：
 - 随机游走概率p, q
 - LINE：
 - 两种损失函数
 - 负样本数量
- 可视化实验：
 - 基于t-SNE对不同模型生成的Node Embedding进行降维可视化

【Node embedding —— Node classification】

- 主要目标：基于Node embedding实现图上的node classification任务
- 数据集：Cora, Citeseer, Pubmed（三个选一个）

- 任务描述：根据给定的图结构数据，建模获得每个节点的Embedding
- 数据集获取&处理：
 - 使用DGL加载三个数据集，分别为：
 - `dgl.data.CoraGraphDataset()`
 - `dgl.data.CiteseerGraphDataset()`
 - `dgl.data.PubmedGraphDataset()`
 - 基于有标签节点集合，将该集合划分成训练集和测试集；
- 主实验：
 - 在整个图结构上获得训练得到每个节点的Node embedding，可使用的DGL接口有：
 - `dgl.sampling.random_walk()`
 - `dgl.sampling.node2vec_random_walk()`
 - 准备一个多分类器（多层神经网络），该分类器的输出域为节点标签的总数，使用训练集节点的embedding和标签训练该分类器
 - 在测试集上测试分类器的预测准确度，评测指标为Accuracy
- 超参实验：
 - Deepwalk：
 - 随机游走路径长度
 - 每个节点起始的路径数量
 - node2vec：
 - 随机游走概率p, q
 - LINE：
 - 两种损失函数
 - 负样本数量
 - 不同种类的多分类器

【GNN——半监督节点分类任务】

- 主要目标：基于GNN实现半监督节点分类任务
- 数据集：Cora, Citeseer, Pubmed（三个选一个）
- 任务描述：给定一个图数据 $G = (V, X, A, Y)$ ，其中V表示节点集合， X 表示节点的属性集合， A 表示图的邻接矩阵， Y 表示节点的标签集合；根据涂上的有标签节点训练GNN，从而为无标签节点预测标签。
- 数据集获取&处理：
 - 使用DGL加载三个数据集，分别为：（注意和【Node embedding】中load数据集不同，这里需要加载节点属性和节点标签）
 - `dgl.data.CoraGraphDataset()`
 - `dgl.data.CiteseerGraphDataset()`
 - `dgl.data.PubmedGraphDataset()`
 - 将图上有标签节点划分为训练集和测试集，节点集合分别定为 V_{train} 和 V_{test} 其标签集合分别定义 Y_{train} 和 Y_{test}
- 主实验：
 - 在 $(X, A, V_{train}, Y_{train})$ 上训练GNN模型，注意测试节点虽然在图结构上，但其标签认为是未知的。

- GCN、GAT、Graphsage的卷积层可以使用dgl.nn下的相应接口
- 测试节点集合 上测试GNN的分类准确度，评测指标为Accuracy
- 比较不同模型在训练集和测试集上的分类准确度
- 超参实验：
 - GNN层数

【GNN——图分类任务 (Graph classification) 】

- 主要目标：基于GNN实现图分类任务
- 数据集：ENZYMES、DD、COLLAB（三个选一个）
- 任务描述：给定一个图，使用GNN预测该图的标签
- 数据集获取&处理：
 - 使用dgl.data导入以上三个数据集
 - 将数据集划分成训练集/测试集
- 主实验：
 - 在训练集上训练GNN（GCN或GAT）+ Pooling（Average Pooling, Mean Pooling和Diff Pooling）模型；
 - 在测试集上验证以上模型的分类效果，评测指标为Accuracy
 - 比较不同GNN模型和不同Pooling 模型组合的分类准确度；
 - 比较不同模型在训练集和测试集上的分类准确度；
- 超参实验：
 - GNN层数

4.6 提交内容

- 理论学习内容包括：
 - Node embedding三种模型的建模过程的数学表示，核心建模思想以及异同
 - GNN基础、不同邻居聚合机制的数学表述以及不同邻居聚合机制的异同
 - 不同Pooling策略以及优劣
 - 半监督任务基础以及任务核心
 - 在使用GNN模型时：Node classification任务和Graph classification任务的异同
- 实验报告

参考资料

Pytorch —— DGL官方文档： <https://docs.dgl.ai/en/0.8.x/api/python/nn-pytorch.html>

GCN 博客： <https://tkipf.github.io/graph-convolutional-networks/>

